# چالش ۳۰ روزهٔ اندروید

آمادگی مصاحبهٔ اندروید — جونیور ← مید ← سینیور ← آرکیتکت

**نسخهٔ ۲۰۲۶**

روز ۱۳

## معماری لایهٔ داده — بخش ۱: repositoryها، مدل‌ها، ذخیره‌سازی و cache



**اسماعیل مرادی**

@esmaeeil-moradi700

# چرا لایهٔ داده روز سیزدهم است؟

روز ۳ repository را به‌عنوان منبع واحد حقیقت معرفی کرد. مصاحبه‌ها خیلی جلوتر می‌روند: *این داده کجا زندگی می‌کند، چقدر تازه است، چه کسی اجازهٔ نوشتنش را دارد، و وقتی خطا رخ داد چه می‌شود؟*

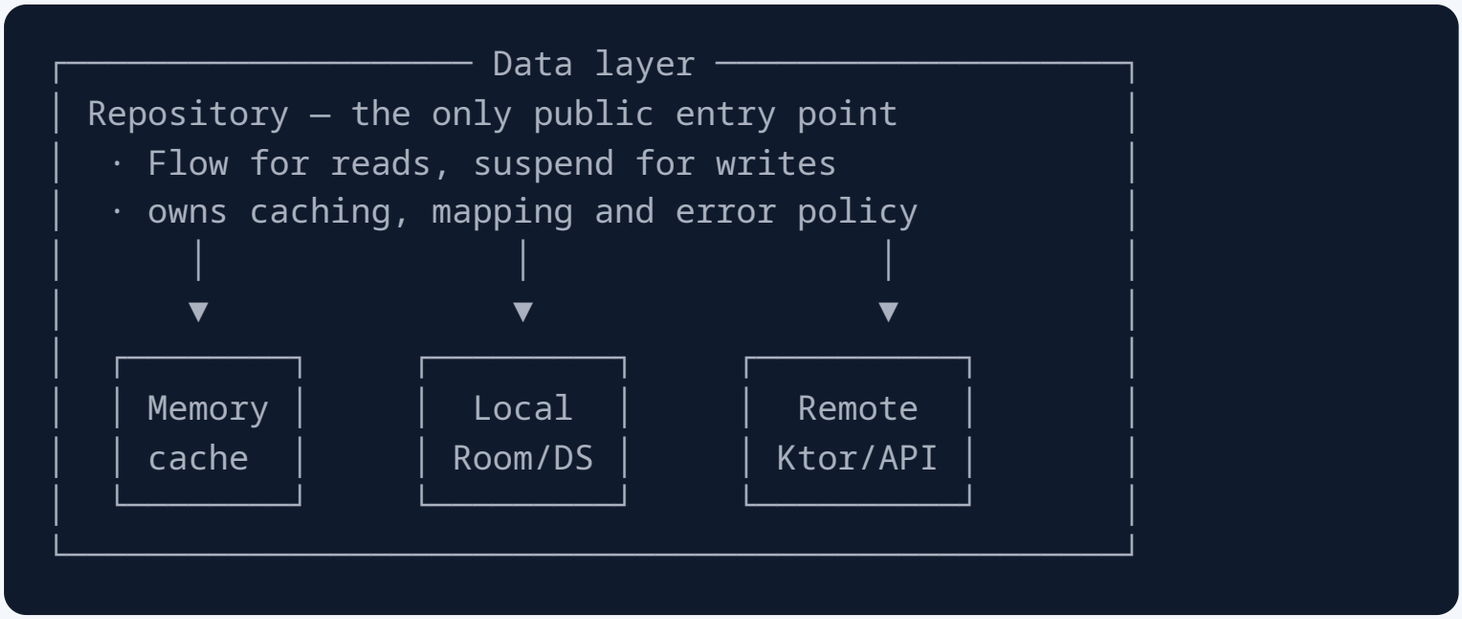
- طراحی repository و data source، که API عمومی داده‌های شما هستند
- مدل جدا برای هر مرز، و تبدیلی که از اپ محافظت می‌کند
- انتخاب محل ذخیره: Room، DataStore، فایل — یا هیچ‌کدام
- استراتژی‌های cache و **invalidation**
- مدیریت خطا و هم‌روندی که در شبکهٔ واقعی دوام بیاورد

| سطح | سؤال‌ها |
| --- | --- |
| جونیور | مسئولیت repository، data sourceها، cache حافظه یا دیسک |
| مید | مدل‌ها و mapping، انتخاب ذخیره‌ساز، استراتژی‌های cache، invalidation |
| سینیور | مدیریت خطا، هم‌روندی در repository، سازگاری داده با چند نویسنده |
| آرکیتکت | APIهای repository در مقیاس بزرگ، سیاست تازگی داده، مرزهای دفاعی |



**اسماعیل مرادی**
@esmaeeil-moradi700

# مدل ذهنی: یک در برای ورود به داده




- **repositoryها** تنها در ورودی‌اند؛ data sourceها داخل لایهٔ داده می‌مانند
- **هر data source** دقیقاً با یک منبع کار می‌کند: یک جدول، یک فایل، یک API
- **سیاست در repository است:** تازگی داده، تبدیل مدل، خطاها، و اینکه چه کسی حق نوشتن دارد

**سؤال پشت همهٔ جواب‌های امروز**

برای هر داده باید بتوانید بگویید: *مالکش کیست، کجا ذخیره می‌شود، چقدر می‌تواند کهنه باشد، و وقتی بارگذاری‌اش شکست خورد کاربر چه می‌بیند؟*



**اسماعیل مرادی**
@esmaeeil-moradi700

# جونیور

مسئولیت repository، data sourceها، cache حافظه یا دیسک



**اسماعیل مرادی**
@esmaeeil-moradi700

# سؤال ۱. repository واقعاً مسئول چیست؟

راهنمای لایهٔ دادهٔ گوگل پنج وظیفه برای repository تعریف می‌کند:

- **منتشر کردن داده** برای بقیهٔ اپ
- **متمرکز کردن تغییرات** داده
- **حل تعارض** بین چند منبع داده
- **پنهان کردن** منابع داده از بقیهٔ اپ
- **نگه‌داشتن منطق بیزینسی** همان داده (مثلاً «پیش‌نویس هرگز آپلود نمی‌شود»)

قاعدهٔ نام‌گذاری: نوع داده و بعد کلمهٔ `Repository`؛ مثل `NewsRepository` و `PaymentsRepository`. یک repository برای هر **نوع داده**، نه برای هر صفحه.



**اسماعیل مرادی**
@esmaeeil-moradi700

# سؤال ۱. چیزهایی که مصاحبه‌گر چک می‌کند

**تلهٔ مصاحبه**

«*repository یک wrapper دور Retrofit است.*» — wrapperی که فقط صدا را پاس می‌دهد هیچ ارزشی ندارد. ارزش در **سیاست** است: کدام منبع برنده است، داده چقدر باید تازه باشد، و خطاها و تعارض‌ها چطور حل می‌شوند.

**تلهٔ مصاحبه**

«*برای هر صفحه یک repository.*» — صفحه‌ها داده را از چند repository ترکیب می‌کنند (در ViewModel یا یک use case). یک `ProfileScreenRepository` منطقی را تکرار می‌کند که باید فقط یک بار در `UserRepository` باشد.

**سؤال تکمیلی**

«*یک repository می‌تواند به repository دیگری وابسته باشد؟*» — بله، وقتی داده لایه‌لایه است (مثلاً `OrdersRepository` که کاربر فعلی را لازم دارد). گراف را بدون چرخه نگه دارید؛ اگر دو repository به هم نیاز دارند، راه‌حل یک use case بالای هر دوست.



**اسماعیل مرادی**
@esmaeeil-moradi700

# سؤال ۲. repositoryها و data sourceها

```
class NewsRemoteDataSource(
    private val api: NewsApi,
) {
    suspend fun fetchLatest(): List<NewsDto> = api.latest()
}

class NewsLocalDataSource(private val dao: NewsDao) {
    fun observeLatest(): Flow<List<NewsEntity>> = dao.observe()
    suspend fun replaceAll(items: List<NewsEntity>) =
        dao.replaceAll(items)
}

class NewsRepository(
    private val remote: NewsRemoteDataSource,
    private val local: NewsLocalDataSource,
) { /* combines both, owns the policy */ }
```

- data source **نازک** است: یک منبع، بدون سیاست، بی‌خبر از منابع دیگر
- نامش نوع داده و بعد نوع منبع است: `NewsRemoteDataSource` و `NewsLocalDataSource`



**اسماعیل مرادی**
@esmaeeil-moradi700

# سؤال ۲. قاعده‌هایی که مرز را تمیز نگه می‌دارند

- ViewModelها و use caseها **فقط با repository** حرف می‌زنند — هرگز با یک DAO یا سرویس API
- data sourceها می‌توانند مدل‌های ذخیره‌سازی را برگردانند (`NewsEntity` و `NewsDto`)؛ repository آن‌ها را تبدیل می‌کند
- interface فقط جایی که به درز نیاز دارید (fake در تست، چند پیاده‌سازی) — نه از روی عادت

**تلهٔ مصاحبه**

«*ViewModel می‌تواند مستقیم DAO را صدا بزند، سریع‌تر است.*» — حالا دو نقطهٔ ورود در یک جدول می‌نویسند، منطق cache و mapping کپی می‌شود، و منبع واحد حقیقت از بین رفته است.

**تلهٔ مصاحبه**

«*هر کلاسی برای تست یک interface لازم دارد.*» — data sourceی که Room را می‌پوشاند اغلب با یک دیتابیس in-memory تست می‌شود؛ interface وقتی ارزش دارد که نسخهٔ واقعی کند، ناپایدار یا راه دور باشد.



**اسماعیل مرادی**
@esmaeeil-moradi700

# سؤال ۳. cache در حافظه یا روی دیسک

| | حافظه (in-memory) | دیسک |
|---|---|---|
| کجا | heap؛ مثل `StateFlow` ، `Map` ، `LruCache` | Room، DataStore، فایل، cache در OkHttp |
| دوام | از configuration change (اگر در سطح اپ باشد) | از process death، ری‌استارت و آپدیت اپ |
| سرعت | سریع‌ترین — بدون I/O | I/O لازم دارد — بیرون از ترد اصلی |
| اندازه | کوچک — با حافظهٔ اپ رقابت می‌کند | بزرگ — محدود به فضای ذخیره‌سازی |
| کاربرد | دادهٔ داغ در طول یک session | هر چیزی که باید آفلاین یا در cold start نشان داده شود |

بیشتر اپ‌ها هر دو را دارند: حافظه برای session فعلی، دیسک به‌عنوان نسخهٔ ماندگار، و شبکه برای تازه‌سازی.



**اسماعیل مرادی**
@esmaeeil-moradi700

# سؤال ۳. جاهایی که cache حافظه خراب می‌شود

**تلهٔ مصاحبه**

«*در ViewModel cache‌ش می‌کنم.*» — آن cache به یک صفحه محدود است و با آن از بین می‌رود. cache حافظهٔ سراسری اپ جایش در یک **repository با scope سینگلتون** است.

**تلهٔ مصاحبه**

نوشتن `LruCache` دستی برای bitmap (توصیهٔ رایج دوران نسخهٔ پارسال). در ۲۰۲۶ کتابخانه‌های تصویر مثل **Coil** یا **Glide** خودشان cache حافظه **و** دیسک را با اندازهٔ درست مدیریت می‌کنند — به‌جایش آن‌ها را تنظیم کنید.

**سؤال تکمیلی**

«*cache حافظه thread-safe است؟*» — فقط اگر خودتان این‌طور بسازیدش. آن را در یک `MutableStateFlow` با `update {}` نگه دارید، یا از یک `Map` با `Mutex` محافظت کنید (سؤال ۹).



**اسماعیل مرادی**
@esmaeeil-moradi700

# میدل

مدل‌ها و mapping، انتخاب ذخیره‌ساز، استراتژی‌های cache، invalidation



**اسماعیل مرادی**
@esmaeeil-moradi700

# سؤال ۴. یک مدل برای هر مرز

```kotlin
@Serializable                          // network contract
data class ArticleDto(
    val id: String,
    val title: String,
    @SerialName("published_at") val publishedAt: String,
)

@Entity(tableName = "articles")        // storage schema
data class ArticleEntity(
    @PrimaryKey val id: String,
    val title: String,
    val publishedAtEpochMs: Long,
)

data class Article(                    // what the app uses
    val id: ArticleId,
    val title: String,
    val publishedAt: Instant,
)
```



اسماعیل مرادی
@esmaeeil-moradi700

# سؤال ۴. mapping — و اینکه کِی لازم نیست

```
fun ArticleDto.toEntity() = ArticleEntity(
    id = id,
    title = title,
    publishedAtEpochMs = Instant.parse(publishedAt)
        .toEpochMilliseconds(),
)

fun ArticleEntity.toDomain() = Article(
    id = ArticleId(id),
    title = title,
    publishedAt = Instant
        .fromEpochMilliseconds(publishedAtEpochMs),
)
```

- هر مدل به **دلیل خودش** تغییر می‌کند: API، اسکیما، یا محصول
- اپ کوچک و بدون منطق در میان؟ شاید دو مدل (DTO و entity) کافی باشد — عمل‌گرا باشید

**تلهٔ مصاحبه**

«*data classهای کاتلین به‌طور پیش‌فرض تغییرناپذیرند*» (نسخهٔ پارسال این را گفته بود). — فقط اگر همهٔ propertyها `val` و از نوع تغییرناپذیر باشند. فیلد `var` یا یک `MutableList` داخل data class کاملاً قابل‌تغییر است.



**اسماعیل مرادی**
@esmaeeil-moradi700

# سؤال ۵. انتخاب ذخیره‌ساز محلی

| داده | انتخاب | چرا |
|---|---|---|
| ساخت‌یافته، قابل query، قابل observe، رابطه‌ای | **Room** | SQL، ایندکس، تراکنش، queryهای `Flow` |
| تنظیمات کوچک، flagها، توکن‌ها | **DataStore** از نوع Preferences | async، تراکنشی، مبتنی بر `Flow` |
| یک شیء کوچک تایپ‌دار | **DataStore** همراه با یک serializer از نوع Proto یا JSON | type-safe و دارای اسکیما |
| تصویر، ویدیو، سند، blobهای بزرگ | **فایل** (`filesDir` و `cacheDir`) | stream کردن؛ مسیر فایل در Room |
| پاسخ‌های HTTP قابل دانلود مجدد | **cache در OkHttp** | رعایت `Cache-Control` (روز ۱۱) |
| key-value قدیمی | `SharedPreferences` | به DataStore مهاجرتش دهید |

سیستم ممکن است وقتی فضا کم است `cacheDir` را پاک کند — هرگز داده‌ای را که نمی‌توانید دوباره بسازید آنجا نگذارید.



**اسماعیل مرادی**
@esmaeeil-moradi700

# سؤال ۵. محدودیت‌های هر گزینه

**تلهٔ مصاحبه**

«*DataStore یک دیتابیس کوچک است.*» — DataStore با هر به‌روزرسانی کل فایلش را بازنویسی می‌کند. توصیهٔ گوگل: برای دادهٔ بزرگ یا پیچیده، **به‌روزرسانی جزئی** یا یکپارچگی ارجاعی، از Room استفاده کنید.

**تلهٔ مصاحبه**

ساختن دو نمونهٔ DataStore برای **یک فایل** در یک پروسس. تضمین‌های DataStore را می‌شکند و هنگام خواندن یا نوشتن فایل exception می‌دهد. یک بار بسازیدش: سینگلتون در DI، یا delegate مربوط به `preferencesDataStore` در سطح بالای فایل.

**تلهٔ مصاحبه**

BLOBهای بزرگ در ردیف‌های Room. ردیف‌هایی که از cursor window بزرگ‌ترند بارگذاری نمی‌شوند و هر query بایت‌ها را کپی می‌کند. فایل را روی دیسک و مسیرش را در جدول نگه دارید.



**اسماعیل مرادی**
@esmaeeil-moradi700

# سؤال ۶. استراتژی‌های cache

| استراتژی | چطور کار می‌کند | مناسب برای |
|---|---|---|
| Stale-while-revalidate | همین حالا cache را نشان بده، در پس‌زمینه تازه کن؛ دیتابیس به‌روزرسانی را emit می‌کند | فید، پروفایل، کاتالوگ |
| اول cache با TTL | تا وقتی cache از *X* جوان‌تر است از آن استفاده کن؛ وگرنه fetch کن | config، دادهٔ مرجع |
| اول شبکه، cache به‌عنوان پشتیبان | شبکه را امتحان کن؛ اگر شکست خورد cache را با برچسب «قدیمی» نشان بده | قیمت، موجودی |
| فقط شبکه | هرگز ذخیره نکن | جست‌وجوی لحظه‌ای، OTP، وضعیت پرداخت |
| فقط cache، پرشده با sync | UI فقط از دیتابیس می‌خواند؛ sync پس‌زمینه در آن می‌نویسد | اپ‌های offline-first (روز ۱۴) |

استراتژی را **برای هر نوع داده** انتخاب کنید، نه برای کل اپ.



**اسماعیل مرادی**
@esmaeeil-moradi700

# سؤال ۶. الگوی stale-while-revalidate با Room به‌عنوان SSOT

```kotlin
class NewsRepository(
    private val local: NewsLocalDataSource,
    private val remote: NewsRemoteDataSource,
    private val clock: Clock,
) {
    fun observeNews(): Flow<List<News>> = local.observeNews()

    suspend fun refreshIfStale(maxAge: Duration = 15.minutes) {
        val last = local.lastFetchedAt()
        if (last != null && clock.now() - last < maxAge) return
        val fresh = remote.fetchNews()
        local.replaceAll(fresh, fetchedAt = clock.now())
    }
}
```

UI تابع `observeNews()` را collect می‌کند؛ ViewModel جداگانه `refreshIfStale()` را صدا می‌زند. ردیف‌های cacheشده فوراً ظاهر می‌شوند و ردیف‌های تازه از **همان** `Flow` می‌رسند. یک `Clock` تزریق‌شده TTL را تست‌پذیر می‌کند.

**تلهٔ مصاحبه**

جریان نسخهٔ پارسال: اول حافظه، بعد دیتابیس، بعد شبکه، و «*دادهٔ تازه را به ViewModel برگردان*». نتیجهٔ یک‌باره‌ای که دیتابیس را دور می‌زند دو مسیر به UI می‌سازد — و همهٔ تغییرات بعدی را از دست می‌دهد.



**اسماعیل مرادی**
@esmaeeil-moradi700

# سؤال ۷. invalidation در cache

| محرک | مثال |
|---|---|
| زمان (TTL) | کاتالوگ قدیمی‌تر از ۱ ساعت ← تازه‌سازی در باز شدن بعدی |
| اقدام کاربر | pull-to-refresh یک fetch اجباری می‌زند |
| سیگنال سرور | پیام push فقط‌داده: «سفارش‌ها تغییر کرد» ← زمان‌بندی تازه‌سازی |
| نوشتن محلی | بعد از ثبت کامنت، رشته را تازه (یا patch) کن |
| تغییر هویت | خروج از حساب یا تعویض حساب ← **پاک کردن** دادهٔ مخصوص کاربر |
| تغییر اسکیما یا قرارداد | آپدیت اپ مدل را عوض می‌کند ← migrate یا حذف cache |
| اندازه | حذف LRU برای cache حافظه و تصویر |

invalidation یعنی علامت زدن داده به‌عنوان کهنه یا حذفش — UI همچنان از همان منبع می‌خواند.



**اسماعیل مرادی**
@esmaeeil-moradi700

# سؤال ۷. خروج از حساب باید همه چیز را پاک کند

```
class UserDataCleaner(
    private val db: AppDatabase,
    private val prefs: DataStore<Preferences>,
    private val httpCache: Cache,
    private val io: CoroutineDispatcher,
) {
    suspend fun wipe() = withContext(io) {
        db.clearAllTables()      // blocking: never on main
        prefs.edit { it.clear() }
        httpCache.evictAll()     // OkHttp disk cache
        // + user files, image caches, scheduled work
    }
}
```

**تلهٔ مصاحبه**

«*خروج از حساب یعنی حذف توکن.*» — حساب بعدی روی همان گوشی سفارش‌ها، تصویرهای cacheشده و پاسخ‌های HTTP کاربر قبلی را می‌بیند. هر ذخیره‌ساز مخصوص کاربر باید پاک شود — یا با شناسهٔ کاربر کلیدگذاری شود.



اسماعیل مرادی
@esmaeeil-moradi700

# سینیور

مدیریت خطا، هم‌روندی در repository، سازگاری داده با چند نویسنده



**اسماعیل مرادی**

@esmaeeil-moradi700

# سؤال ۸. مدیریت خطا در سراسر لایهٔ داده

```
sealed interface DataError {
    data object Offline : DataError
    data class Http(val code: Int) : DataError
    data object BadData : DataError
}

suspend fun <T> safeCall(block: suspend () -> T): Outcome<T> =
    try {
        Outcome.Ok(block())
    } catch (e: CancellationException) {
        throw e
    } catch (e: IOException) {
        Outcome.Err(DataError.Offline)
    } catch (e: HttpException) {
        Outcome.Err(DataError.Http(e.code()))
    } catch (e: SerializationException) {
        Outcome.Err(DataError.BadData)
    }
```

exceptionهای data source پرتاب می‌کنند؛ repository آن‌ها را به خطاهایی **ترجمه** می‌کند که اپ می‌فهمد. صدازننده‌ها هرگز تایپ‌های Retrofit، Ktor یا SQLite را import نمی‌کنند.



**اسماعیل مرادی**
@esmaeeil-moradi700

# سؤال ۸. قاعده‌های خطا

- دسته‌بندی کنید: **گذرا** (آفلاین، timeout، خطای 5xx ← بعداً دوباره)، **دائمی** (خطای اعتبارسنجی 4xx ← به کاربر بگو)، **احراز هویت** (401 ← تازه‌سازی توکن، روز ۱۲)
- تازه‌سازی شکست خورد اما cache دارید؟ داده را نشان دهید **به‌علاوهٔ** یک پیام — نه یک صفحهٔ خطای کامل
- عملگر `catch` روی `Flow` فقط خطاهای **بالادستی** را می‌گیرد، و flow بعد از آن تمام می‌شود

**تلهٔ مصاحبه**

«*`Result` کاتلین نمی‌تواند نوع خروجی تابع باشد.*» — قدیمی است: این محدودیت در Kotlin 1.5 برداشته شد. سؤال واقعی این است که آیا `Throwable` به اندازهٔ کافی دقیق است — معمولاً یک تایپ خطای sealed واضح‌تر است.

**تلهٔ مصاحبه**

«*هر درخواست ناموفق را دوباره بفرست.*» — تکرار یک `POST` غیرِ idempotent می‌تواند دو سفارش بسازد. فقط فراخوانی‌های idempotent را تکرار کنید، یا آن‌هایی که idempotency key دارند (روز ۱۴).



**اسماعیل مرادی**
@esmaeeil-moradi700

# سؤال ۹. هم‌روندی داخل repository

```
class ProfileRepository(
    private val remote: ProfileRemoteDataSource,
    private val externalScope: CoroutineScope, // app-scoped
) {
    private val mutex = Mutex()
    private var cached: Profile? = null

    suspend fun profile(refresh: Boolean = false): Profile {
        if (!refresh) {
            mutex.withLock { cached }?.let { return it }
        }
        return externalScope.async {
            val fresh = remote.fetch()
            mutex.withLock { cached = fresh }
            fresh
        }.await()
    }
}
```

- `Mutex` از cache محافظت می‌کند؛ قفل در طول درخواست شبکه نگه داشته **نمی‌شود**
- الگوی `externalScope.async { }.await()`: اگر صفحه بسته شود، صدازننده لغو می‌شود اما fetch تمام می‌شود و **cache را پر می‌کند** (الگوی راهنمای لایهٔ دادهٔ گوگل)



**اسماعیل مرادی**
@esmaeeil-moradi700

# سؤال ۹. چیزهایی که مصاحبه‌گر می‌کاود

- **main-safety:** هیچ repositoryی صدازننده را block نمی‌کند. APIهای `suspend` در Room، Retrofit و Ktor خودشان main-safe هستند
- **کاری که باید تمام شود** (ذخیرهٔ پیش‌نویس، درخواست پرداخت): آن را در یک `CoroutineScope` سطح اپ که DI تزریق می‌کند اجرا کنید — یا WorkManager، اگر باید از process death عبور کند
- **درخواست‌های تکراری:** ده صدازننده یعنی ده fetch. یک `Deferred` در حال اجرا را به اشتراک بگذارید، یا تازه‌سازی را از یک نقطه انجام دهید

**تلهٔ مصاحبه**

استفاده از `GlobalScope.launch` برای نوشتن‌های «بفرست و فراموش کن». بدون مالک، بدون لغو، تست‌ناپذیر. یک `CoroutineScope` سطح اپ تزریق کنید تا تست بتواند کنترلش کند.

**تلهٔ مصاحبه**

پیچیدن کل عملیات در `withContext(NonCancellable)`. لغو را همه‌جا پنهان می‌کند؛ فقط برای cleanup کوتاهی استفاده‌اش کنید که باید بعد از لغو اجرا شود.



**اسماعیل مرادی**
@esmaeeil-moradi700

# سؤال ۱۰. سازگاری داده وقتی چند نویسنده به یک جدول دست می‌زنند

```
@Dao
abstract class ArticleDao {
    @Query("DELETE FROM articles WHERE feed = :feed")
    abstract suspend fun deleteFeed(feed: String)

    @Upsert
    abstract suspend fun upsertAll(items: List<ArticleEntity>)

    @Transaction
    open suspend fun replaceFeed(
        feed: String,
        items: List<ArticleEntity>,
    ) {
        deleteFeed(feed)
        upsertAll(items)
    }
}
```

یک تراکنش یعنی observerها هرگز جدول خالیِ وسط کار را نمی‌بینند، و crash دادهٔ قبلی را سالم باقی می‌گذارد.



**اسماعیل مرادی**
@esmaeeil-moradi700

# سؤال ۱۰. رقابت بین شبکه، کاربر و sync

- **پاسخ کهنه روی ویرایش جدیدتر می‌نویسد:** نوشتن را با یک version محافظت کنید، مثلاً با شرط `WHERE version < :newVersion`
- **تازه‌سازی در برابر ویرایش محلی:** ردیف‌هایی را که هنوز تغییر محلیِ ارسال‌نشده دارند بازنویسی نکنید (روز ۱۴: outbox)
- Room یک query مشاهده‌شده را وقتی **جدول خودش** تغییر کند دوباره اجرا می‌کند — با `distinctUntilChanged()` از emitهای تکراری جلوگیری کنید

**تلهٔ مصاحبه**

«*به‌روزرسانی Room و فراخوانی شبکه را در یک تراکنش بپیچید*» (نسخهٔ پارسال). — غیرممکن است: تراکنش دیتابیس نمی‌تواند سرور راه دور را در بر بگیرد. باز نگه داشتن تراکنش نوشتن در طول I/O شبکه هم همهٔ نویسنده‌های دیگر را block می‌کند. اول محلی commit کنید، بعد sync.



**اسماعیل مرادی**
@esmaeeil-moradi700

# آرکیتکت

APIهای repository در مقیاس بزرگ، سیاست تازگی داده، مرزهای دفاعی



**اسماعیل مرادی**
@esmaeeil-moradi700

# سؤال ۱۱. طراحی API برای repository در مقیاس بزرگ

```kotlin
// :core:data — the public contract
interface NewsRepository {
    fun observeNews(topic: TopicId): Flow<List<News>>
    suspend fun refresh(topic: TopicId): Outcome<Unit>
    suspend fun setBookmarked(id: NewsId, saved: Boolean)
}

// Implementation and data sources stay module-private
internal class OfflineFirstNewsRepository(
    private val dao: NewsDao,
    private val api: NewsApi,
) : NewsRepository { /* … */ }
```

چیدمان ماژول‌ها به سبک Now in Android: ماژول‌های `:core:network`، `:core:database` و `:core:datastore` زیر `:core:data` قرار می‌گیرند و فیچرها فقط به `:core:data` وابسته‌اند. فیچرها interface را می‌بینند، هرگز DAO را.



**اسماعیل مرادی**
@esmaeeil-moradi700

# سؤال ۱۱. تصمیم‌های پشت API

| تصمیم | ترجیح | پرهیز |
| --- | --- | --- |
| دانه‌بندی | یک repository برای هر نوع داده | یکی برای هر صفحه؛ یک repository همه‌کاره |
| خواندن | `Flow` از مدل‌های domain | تابع `suspend get()` برای داده‌ای که تغییر می‌کند |
| نوشتن | تابع `suspend` که نتیجه برمی‌گرداند | «بفرست و فراموش کن» از scope لایهٔ UI |
| مالکیت | هر جدول را فقط یک repository می‌نویسد | دو repository که روی همان ردیف‌ها می‌نویسند |
| شکل | فعل‌های domain مثل `setBookmarked()` | یک `BaseRepository<T>` عمومی با CRUD |

**تلهٔ مصاحبه**

یک `BaseRepository<T>` عمومی با `getAll/insert/delete`. معنای ذخیره‌سازی را به domain نشت می‌دهد و با هر قاعده‌ای که CRUD نیست می‌جنگد (پیش‌نویس، حذف نرم، دسترسی‌ها).



**اسماعیل مرادی**
@esmaeeil-moradi700

# سؤال ۱۲. تازگی داده یک تصمیم محصولی است

| داده | سیاست | کاربر می‌بیند |
|---|---|---|
| feature flag و config | اول cache، TTL طولانی، تازه‌سازی در شروع | آخرین config شناخته‌شده |
| فید و کاتالوگ | Stale-while-revalidate | محتوای فوری، به‌روزرسانی بی‌صدا |
| پروفایل و تنظیمات | اول محلی، نوشتن‌ها sync می‌شوند | ویرایش‌های خودش را فوراً |
| موجودی حساب، انبار، قیمت | اول شبکه، cache به‌عنوان پشتیبان | مقدار **همراه با زمانش** («تا ساعت ۱۰:۴۲») |
| نتایج جست‌وجو | فقط حافظه | هیچ چیز ذخیره نمی‌شود |
| رمزها و OTP | هرگز cache نمی‌شوند | — |

- سیاست را **برای هر نوع داده** بنویسید (یک ADR یا جدولی در ریپو)، بعد در repository پیاده‌اش کنید
- نشان دادن دادهٔ قدیمی اشکالی ندارد وقتی **بگویید** قدیمی است؛ پول یا موجودیِ بی‌صدا کهنه یک باگ است

**تلهٔ مصاحبه**

«*همه چیز را offline-first کن.*» — بعضی داده‌ها نباید بدون برچسب کهنه نشان داده شوند، و بعضی هرگز نباید ذخیره شوند. offline-first برای هر نوع داده یک انتخاب است (روز ۱۴).



**اسماعیل مرادی**
@esmaeeil-moradi700

# سؤال ۱۳. مرزهای دفاعی: قرارداد عوض خواهد شد

```kotlin
@Serializable
data class OrderDto(
    val id: String? = null,
    val status: String? = null,
    val totalCents: Long? = null,
)

fun OrderDto.toDomainOrNull(): Order? {
    val id = id ?: return null
    val total = totalCents ?: return null
    val status = when (status) {
        "pending" -> OrderStatus.Pending
        "paid" -> OrderStatus.Paid
        else -> OrderStatus.Unknown     // new server value
    }
    return Order(OrderId(id), status, Money(total))
}
```

با `dtos.mapNotNull { it.toDomainOrNull() }` یک آیتم خراب حذف (و log) می‌شود، نه کل صفحه.



**اسماعیل مرادی**
@esmaeeil-moradi700

# سؤال ۱۳. نسخه‌های قدیمی اپ سال‌ها زنده می‌مانند

- نسخه‌های قدیمی مدت‌ها بعد از انتشار نسخهٔ جدید نصب می‌مانند: هر تغییر API باید برای آن‌ها **سازگار با گذشته** باشد
- mapper یک **لایهٔ ضدفساد (anti-corruption layer)** است: domain هرگز مقدار nullable بی‌معنی یا enum ناشناخته نمی‌بیند
- دادهٔ ذخیره‌شده هم قرارداد است: ردیف‌های Room، فایل‌های DataStore، ورودی WorkManager و payloadهای صف که نسخهٔ ۱ نوشته، نسخهٔ ۲ می‌خواند (روز ۱۴: migration)

**تلهٔ مصاحبه**

یک enum سخت‌گیر در DTO. کتابخانهٔ kotlinx.serialization به‌طور پیش‌فرض روی مقدار enum ناشناخته exception می‌دهد — یک وضعیت جدید در سرور، parse کل لیست را در همهٔ نصب‌های قدیمی خراب می‌کند. از `String` تبدیل کنید، یا مقدار پیش‌فرض بگذارید.

**جواب سینیور در یک جمله**

«repositoryها تنها در ورود به داده‌اند؛ هر نوع داده یک مالک، یک سیاست تازگی صریح و مدل‌های خودش را دارد، و خطاها در مرز ترجمه می‌شوند.»



**اسماعیل مرادی**
@esmaeeil-moradi700

# روز ۱۳ — خلاصهٔ یک‌صفحه‌ای

- repository داده را منتشر می‌کند، تغییرات را متمرکز می‌کند و مالک سیاست است. **یکی برای هر نوع داده**، نه هر صفحه.
- data sourceها نازک‌اند: یک منبع، بدون سیاست، نامرئی بیرون از لایهٔ داده.
- cache حافظه برای session (در سطح اپ)، دیسک برای آفلاین؛ تصویرها با Coil یا Glide.
- مدل‌های DTO، entity و domain را وقتی به دلایل متفاوت تغییر می‌کنند جدا کنید؛ data class لزوماً تغییرناپذیر نیست.
- Room برای دادهٔ ساخت‌یافته، DataStore برای تنظیمات کوچک، فایل برای blobها.
- استراتژی cache برای هر نوع داده؛ UI همیشه از **یک** منبع می‌خواند.
- invalidation با TTL، اقدام کاربر، سیگنال سرور و نوشتن محلی — و در خروج از حساب **همه چیز را پاک کنید**.
- exceptionها را در repository ترجمه کنید؛ `CancellationException` را دوباره throw کنید؛ فقط idempotentها را تکرار کنید.
- کار سطح اپ در `externalScope` تزریق‌شده؛ cache با `Mutex`؛ شبکه هرگز داخل تراکنش دیتابیس.
- دفاعی تبدیل کنید: مقدار ناشناخته نباید نسخه‌های قدیمی را crash کند.

**روز ۱۴ در راه است: لایهٔ داده بخش ۲ — offline-first، sync و مقیاس.** این پست را ذخیره کنید و همراه ما باشید.



**اسماعیل مرادی**
@esmaeeil-moradi700